Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Чувашский государственный университет им. И.Н. Ульянова»
Кафедра вычислительной техники

Лабораторная работа № 9

Директивы оператора

По дисциплине «Операционные системы»

Выполнил: студент группы ИВТ-42-23
Иванов И.Л.

Проверил: кандидат технических наук
Ковалев С.В.

Чебоксары, 2026

Цель работы

Разработать язык управления моделью ОС: управляющие директивы оператора, изменяющие состояние модели, параметры заданий, память и отдельные процессы.
Исходники: https://github.com/kukirmash/os_model/

Задание

- разработать язык управления моделью;
- реализовать операторы и подпрограммы выполнения директив;
- обеспечить паузу/возобновление автозагрузки, ручную загрузку задания, изменение параметров следующего задания;
- реализовать приостановку/возобновление и уничтожение выбранного процесса;
- реализовать изменение размера памяти и режим «выполнить загруженные и выключить»;
- добавить запуск генератора случайных чисел с заданным seed из командной строки.

В отчёте представить блок-схемы алгоритмов, анализ программного кода, листинг программы и экранные формы.

Краткая характеристика решения

Директивы передаются в ядро через буферизованный канал. Каждая директива содержит вид, идентификатор процесса или параметры задания/памяти. Результат выполнения сохраняется в LastDirective и отображается в интерфейсе. Для управления процессами добавлено состояние «Приостановлен»; уничтожение процесса не засчитывается как выполнение. Seed генератора можно переопределить ключом -seed.

1. Язык управления

Директива	Параметры	Действие	Клавиша
Выключить питание	—	немедленное завершение	Q

Пауза / продолжить модель	—	остановка/возобновление тактов	Space
Изменить скорость	±10 %	изменение скорости	+ / −
Пауза автозагрузки	—	AutoLoadEnabled = false	A
Продолжить автозагрузку	—	AutoLoadEnabled = true	A
Загрузить задание	—	ручная загрузка	L
Параметры следующего задания	размер, команды, ВВ %, приоритет	разовое переопределение	форма
Сбросить параметры задания	—	генерация по params.json	кнопка
Приостановить процесс	ID процесса	Готов/Активен → Приостановлен	P
Активизировать процесс	ID процесса	Приостановлен → Готов	P
Уничтожить процесс	ID процесса	выгрузка без зачёта выполнения	Delete
Выполнить и выключить	—	ожидание пустой таблицы PSW, затем Quit	F
Изменить память	новый объём	изменение Memory.TotalSize	форма
Задать seed RNG	целое число	переопределение seed при старте	-seed

Процесс выбирается щелчком по строке таблицы PSW. Выбранная строка подсвечивается, а в панели директив отображаются номер и состояние процесса.

2. Блок-схемы алгоритмов

Блок-схемы выполнены в чёрно-белом оформлении по ГОСТ 19.701-90: терминатор — скруглённый прямоугольник, процесс — прямоугольник, решение — ромб, состояния — прямоугольники.



Рисунок 1 — Общий цикл обработки директив

HandleDirectives извлекает директивы из канала до его опустошения. Для каждой директивы вызывается обработчик, результат записывается в LastDirective, после чего обновляется индикация.

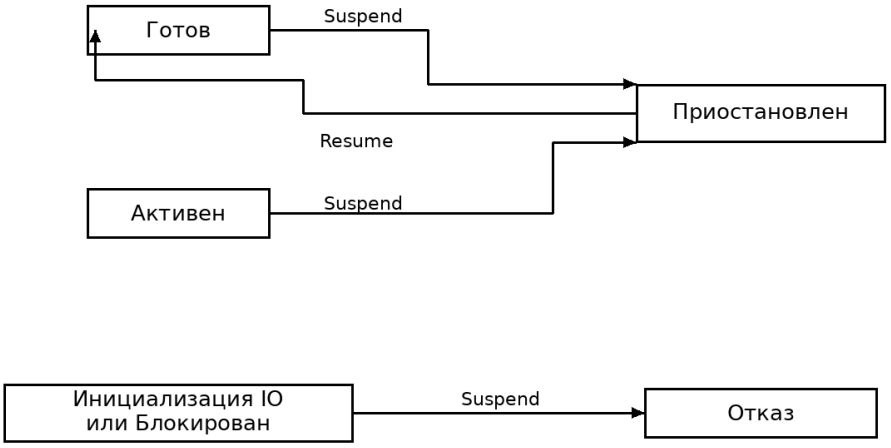


Рисунок 2 — Состояния процесса при приостановке и возобновлении

Приостановить можно процесс в состоянии «Готов» или «Активен». Если процесс активен, центральный процессор освобождается. Процесс в состоянии «Инициализация IO» или «Блокирован» приостановить нельзя.

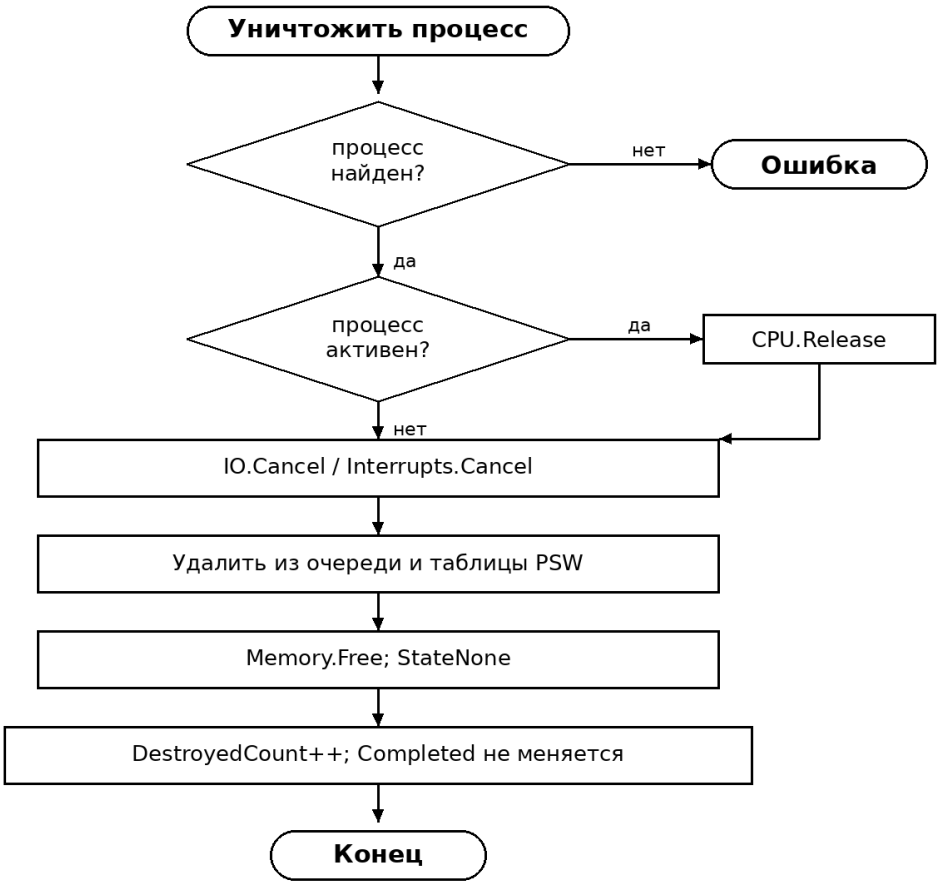


Рисунок 3 — Уничтожение выбранного процесса

При уничтожении процесса отменяются операции ВВ и ожидающие прерывания, освобождаются ЦПр и память, процесс удаляется из таблицы PSW. Счётчик Completed не увеличивается, используется отдельный DestroyedCount.

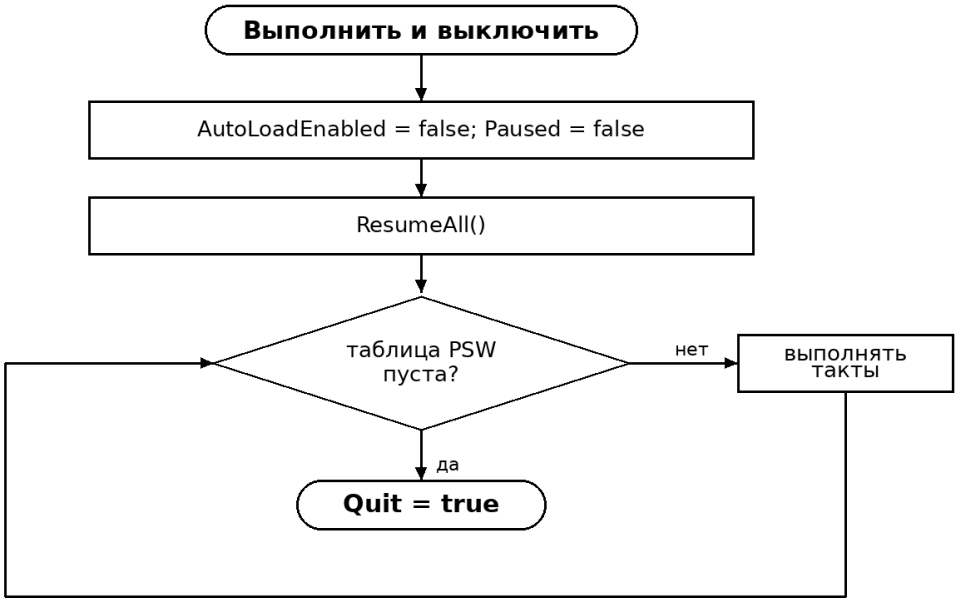


Рисунок 4 — Выполнить загруженные задания и выключить

Директива отключает автозагрузку, снимает паузу, возобновляет приостановленные процессы и завершает модель, когда таблица PSW становится пустой.

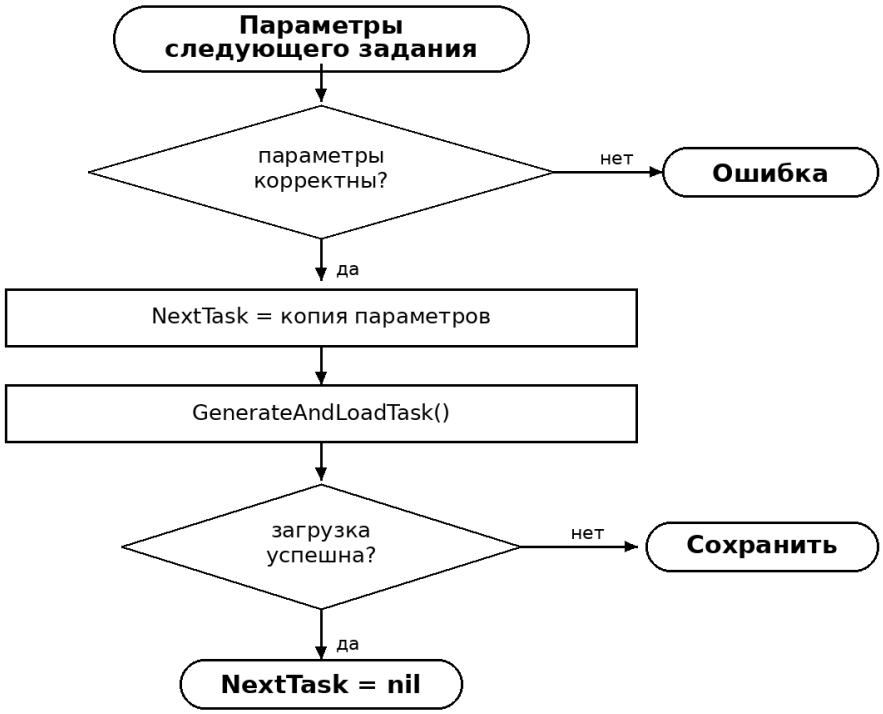


Рисунок 5 — Параметры следующего задания и загрузка

Параметры следующего задания проверяются и сохраняются в NextTask. При успешной загрузке переопределение сбрасывается; при неудаче сохраняется для повторной попытки.

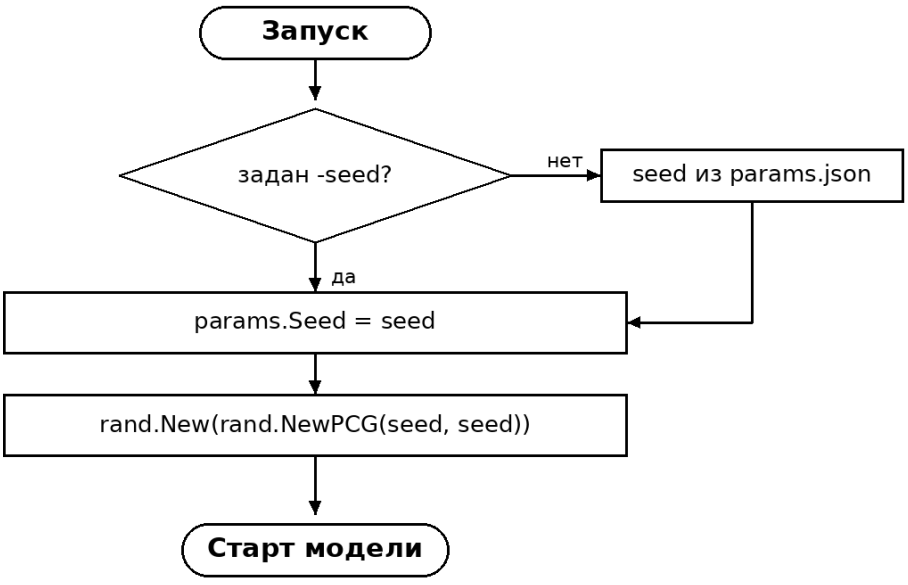


Рисунок 6 — Запуск с ключом -seed

Ключ -seed имеет приоритет над seed из params.json. Один и тот же seed обеспечивает повторяемость параметров, заданий, команд и длительностей ВВ.

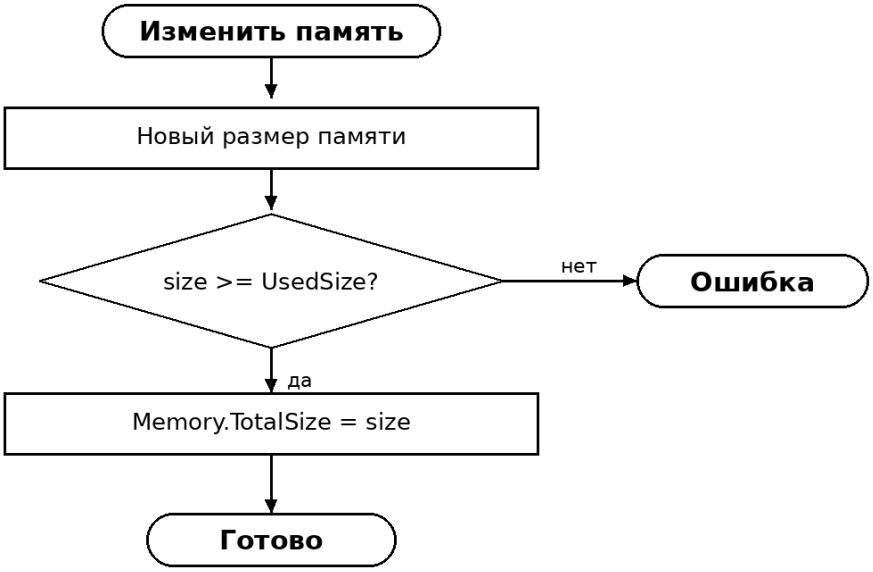


Рисунок 7 — Изменение размера памяти

Новый объём памяти должен быть не меньше уже занятого. При успехе обновляются Memory.TotalSize и Config.MemorySize.

3. Анализ программного кода

3.1. Модель директив

Директивы описаны структурой Directive с видом, идентификатором процесса, параметрами задания и памяти. Результат выполнения сохраняется в DirectiveResult. Канал директив имеет буфер 64, чтобы пачка команд из интерфейса не терялась.

internal/os/directives.go

```
package os

import (
	"fmt"

	"os_model/internal/process"
	"os_model/internal/regulator"
)

// ---------------------------------------------------------------------------
// DirectiveKind — вид управляющей директивы оператора (лабораторная работа 9).
type DirectiveKind int

const (
	DirectivePause     DirectiveKind = iota // пауза моделирования
	DirectiveResume                         // продолжить моделирование
	DirectiveSpeedUp                        // увеличить скорость
	DirectiveSpeedDown                      // уменьшить скорость
	DirectiveQuit                           // выключить питание

	DirectiveAutoLoadPause  // приостановить автоматическую загрузку
	DirectiveAutoLoadResume // продолжить автоматическую загрузку
	DirectiveLoadTask       // загрузить новое задание
	DirectiveSetNextTask    // задать параметры следующего задания
	DirectiveClearNextTask  // сбросить переопределение параметров
	DirectiveSuspendProcess // приостановить выбранный процесс
	DirectiveResumeProcess  // активизировать выбранный процесс
	DirectiveDestroyProcess // уничтожить выбранный процесс
	DirectiveFinishAndQuit  // выполнить загруженные задания и выключить
	DirectiveSetMemory      // изменить размер памяти
)

// ---------------------------------------------------------------------------
// String возвращает имя директивы для индикации.
func (k DirectiveKind) String() string {
	switch k {
	case DirectivePause:
		return "пауза модели"
	case DirectiveResume:
		return "продолжить модель"
	case DirectiveSpeedUp:
		return "увеличить скорость"
	case DirectiveSpeedDown:
		return "уменьшить скорость"
	case DirectiveQuit:
		return "выключить питание"
```

```
	case DirectiveAutoLoadPause:
		return "пауза автозагрузки"
	case DirectiveAutoLoadResume:
		return "возобновить автозагрузку"
	case DirectiveLoadTask:
		return "загрузить задание"
	case DirectiveSetNextTask:
		return "параметры следующего задания"
	case DirectiveClearNextTask:
		return "сбросить параметры задания"
	case DirectiveSuspendProcess:
		return "приостановить процесс"
	case DirectiveResumeProcess:
		return "активизировать процесс"
	case DirectiveDestroyProcess:
		return "уничтожить процесс"
	case DirectiveFinishAndQuit:
		return "выполнить и выключить"
	case DirectiveSetMemory:
		return "изменить память"
	default:
		return "неизвестная директива"
	}
}

// ----------------------------------------------------------------------------
// Directive — директива оператора вместе с параметрами.
type Directive struct {
	Kind       DirectiveKind
	ProcessID  int
	Task       *process.Task
	MemorySize int
}

// ----------------------------------------------------------------------------
// DirectiveResult — результат выполнения последней директивы.
type DirectiveResult struct {
	Time    int    `json:"time"`
	Name    string `json:"name"`
	OK      bool   `json:"ok"`
	Message string `json:"message"`
}

// ----------------------------------------------------------------------------
// applyDirective выполняет одну директиву и запоминает результат.
func (sys *System) applyDirective(d Directive) {
	result := DirectiveResult{
		Time: sys.Time,
		Name: d.Kind.String(),
		OK:   true,
	}

	switch d.Kind {
	case DirectivePause:
		sys.Paused = true
		result.Message = "моделирование приостановлено"

	case DirectiveResume:
		sys.Paused = false
		result.Message = "моделирование продолжено"

	case DirectiveSpeedUp:
```

```
		sys.changeSpeed(speedStepPercent)
		result.Message = fmt.Sprintf("скорость %.1f такт/с", sys.Speed)

	case DirectiveSpeedDown:
		sys.changeSpeed(-speedStepPercent)
		result.Message = fmt.Sprintf("скорость %.1f такт/с", sys.Speed)

	case DirectiveQuit:
		sys.Quit = true
		result.Message = "питание выключено"

	case DirectiveAutoLoadPause:
		sys.AutoLoadEnabled = false
		result.Message = "автоматическая загрузка приостановлена"

	case DirectiveAutoLoadResume:
		sys.AutoLoadEnabled = true
		result.Message = "автоматическая загрузка возобновлена"

	case DirectiveLoadTask:
		if sys.GenerateAndLoadTask() {
			result.Message = fmt.Sprintf("задание #%d загружено", sys.LastTask.Task.ID)
		} else {
			result.OK = false
			result.Message = "задание не загружено: " + sys.LastTask.Reason
		}

	case DirectiveSetNextTask:
		if d.Task == nil {
			result.OK = false
			result.Message = "не заданы параметры задания"
			break
		}
		if err := validateTask(*d.Task); err != nil {
			result.OK = false
			result.Message = err.Error()
			break
		}
		task := *d.Task
		sys.NextTask = &task
		result.Message = fmt.Sprintf("следующее задание: размер %d, команд %d, ВВ %d %%,
приоритет %d",
			task.Size, task.TotalCommands, task.IOPercent, task.BasePriority)

	case DirectiveClearNextTask:
		sys.NextTask = nil
		result.Message = "параметры следующего задания сброшены"

	case DirectiveSuspendProcess:
		if err := sys.suspendProcess(d.ProcessID); err != nil {
			result.OK = false
			result.Message = err.Error()
		} else {
			result.Message = fmt.Sprintf("процесс #%d приостановлен", d.ProcessID)
		}

	case DirectiveResumeProcess:
		if err := sys.resumeProcess(d.ProcessID); err != nil {
			result.OK = false
			result.Message = err.Error()
		} else {
			result.Message = fmt.Sprintf("процесс #%d возобновлён", d.ProcessID)
```

```
                }

        case DirectiveDestroyProcess:
                if err := sys.destroyProcess(d.ProcessID); err != nil {
                        result.OK = false
                        result.Message = err.Error()
                } else {
                        result.Message = fmt.Sprintf("процесс #%d уничтожен", d.ProcessID)
                }

        case DirectiveFinishAndQuit:
                sys.FinishAndQuit = true
                sys.AutoLoadEnabled = false
                sys.Paused = false
                sys.Regulator.ResumeAll()
                result.Message = "загруженные задания выполняются, новые не загружаются"

        case DirectiveSetMemory:
                if err := sys.Memory.SetTotalSize(d.MemorySize); err != nil {
                        result.OK = false
                        result.Message = err.Error()
                } else {
                        sys.Config.MemorySize = d.MemorySize
                        result.Message = fmt.Sprintf("размер памяти: %d единиц", d.MemorySize)
                }

        default:
                result.OK = false
                result.Message = "неизвестная директива"
        }

        sys.LastDirective = result
        sys.notify()
}

// ---------------------------------------------------------------------------
// validateTask проверяет параметры задания, заданные оператором.
func validateTask(task process.Task) error {
        if task.Size <= 0 {
                return fmt.Errorf("размер задания должен быть больше 0")
        }
        if task.TotalCommands <= 0 {
                return fmt.Errorf("число команд должно быть больше 0")
        }
        if task.IOPercent < 0 || task.IOPercent > 100 {
                return fmt.Errorf("процент команд ВВ должен быть в пределах 0..100")
        }
        if task.BasePriority <= 0 {
                return fmt.Errorf("базовый приоритет должен быть больше 0")
        }
        return nil
}

// ---------------------------------------------------------------------------
// findProcess ищет резидентный процесс по идентификатору.
func (sys *System) findProcess(id int) *process.PSW {
        for _, p := range sys.Regulator.Table {
                if p.ID == id {
                        return p
                }
        }
        return nil
```

```
}

// ---------------------------------------------------------------------------
// suspendProcess приостанавливает готовый или активный процесс.
func (sys *System) suspendProcess(id int) error {
	p := sys.findProcess(id)
	if p == nil {
		return fmt.Errorf("процесс #%d не найден", id)
	}
	if p.State != process.StateReady && p.State != process.StateActive {
		return fmt.Errorf("процесс #%d нельзя приостановить в состоянии %s",
			id, process.StateName(p.State))
	}
	if !sys.Regulator.Process(p, regulator.EventSuspend) {
		return fmt.Errorf("не удалось приостановить процесс #%d", id)
	}
	if sys.CPU.ActiveProcess == p {
		sys.CPU.Release()
	}
	return nil
}

// ---------------------------------------------------------------------------
// resumeProcess возвращает приостановленный процесс в очередь готовности.
func (sys *System) resumeProcess(id int) error {
	p := sys.findProcess(id)
	if p == nil {
		return fmt.Errorf("процесс #%d не найден", id)
	}
	if p.State != process.StateSuspended {
		return fmt.Errorf("процесс #%d не приостановлен", id)
	}
	if !sys.Regulator.Process(p, regulator.EventResume) {
		return fmt.Errorf("не удалось возобновить процесс #%d", id)
	}
	return nil
}

// ---------------------------------------------------------------------------
// destroyProcess уничтожает процесс, освобождая ЦПр, ВВ и память.
func (sys *System) destroyProcess(id int) error {
	p := sys.findProcess(id)
	if p == nil {
		return fmt.Errorf("процесс #%d не найден", id)
	}

	if sys.CPU.ActiveProcess == p {
		sys.CPU.Release()
	}
	sys.IO.Cancel(p)
	sys.Interrupts.Cancel(p)
	if !sys.Regulator.Destroy(p) {
		return fmt.Errorf("не удалось уничтожить процесс #%d", id)
	}
	return nil
}

// ---------------------------------------------------------------------------
// checkFinish завершает моделирование, когда все загруженные задания
// выполнены или уничтожены.
func (sys *System) checkFinish() {
	if sys.FinishAndQuit && len(sys.Regulator.Table) == 0 {
```

```
                sys.Quit = true
        }
}
```

3.2. Регулировщик и процессы

В регулировщик добавлены события EventSuspend и EventResume, методы Destroy и ResumeAll, счётчик Destroyed. Уничтожение не вызывает OnComplete, поэтому уничтоженный процесс не попадает в средние значения завершённых заданий.

```
case EventSuspend:
    if p.State != process.StateReady && p.State != process.StateActive {
        return false
    }
    if p.State == process.StateReady {
        r.Planner.Remove(p)
    }
    p.State = process.StateSuspended

case EventResume:
    if p.State != process.StateSuspended {
        return false
    }
    p.State = process.StateReady
    r.Planner.Add(p)
```

3.3. Отмена ВВ и прерываний

При уничтожении процесса нужно убрать его операцию ВВ и ожидающие сигналы прерываний. Для этого добавлены методы Processors.Cancel и Supervisor.Cancel. Это исключает обработку прерывания для уже удалённого процесса.

```
func (m *Processors) Cancel(p *process.PSW) {
    busy := m.Operations[:0]
    for _, op := range m.Operations {
        if op.Process != p {
            busy = append(busy, op)
        }
    }
    m.Operations = busy
}

func (s *Supervisor) Cancel(p *process.PSW) {
    pending := s.Pending[:0]
    for _, signal := range s.Pending {
        if signal.Process != p {
            pending = append(pending, signal)
        }
    }
    s.Pending = pending
}
```

3.4. Учёт приостановленного времени

В PSW добавлено поле SuspendedTicks. В Stats.Collect состояние «Приостановлен» увеличивает этот счётчик, а MultiTicks включает его в T_multi. Инвариант времени процесса:

```
WaitTicks + ExecTicks + IOTicks + SuspendedTicks = Time - LoadedAt
```

3.5. Параметры задания, память и завершение

Параметры следующего задания хранятся в NextTask. GenerateAndLoadTask использует их вместо генерации по диапазонам и очищает после успешной загрузки.

```
if sys.NextTask != nil {
   task = *sys.NextTask
} else {
   task = sys.generateTask()
}

loaded := sys.LoadTask(task)
if loaded && sys.NextTask != nil {
   sys.NextTask = nil
}
```

Размер памяти изменяется через Memory.SetTotalSize с проверкой занятого объёма. Режим FinishAndQuit отключает автозагрузку, возобновляет приостановленные процессы и завершает модель при пустой таблице PSW.

3.6. Приём директив из интерфейса

Wails-событие os:directive теперь передаёт объект с именем и параметрами. app.go поддерживает и старый строковый формат, и JSON-объект. Это позволяет передавать ID процесса, параметры задания и размер памяти.

```
type directiveMessage struct {
   Name       string `json:"name"`
   ProcessID  int    `json:"processId"`
   Size       int    `json:"size"`
   Commands   int    `json:"commands"`
   IOPercent  int    `json:"ioPercent"`
   Priority   int    `json:"priority"`
   MemorySize int    `json:"memorySize"`
}
```

3.7. Интерфейс

В интерфейс добавлена панель «Директивы оператора» с группами загрузки, параметров следующего задания, памяти и выбранного процесса. Таблица PSW стала интерактивной: строка выбирается щелчком и подсвечивается. Состояние «Приостановлен» отображается отдельным цветом.

```
function uiSendDirective(name, payload = {}) {
  if (window.runtime) {
    window.runtime.EventsEmit("os:directive", { name, ...payload });
  }
}
```

4. Листинг основных модулей

Ниже приведён полный листинг модуля директив и ключевые изменения регулировщика. Полный листинг проекта доступен в репозитории.

internal/os/directives.go

```
package os

import (
	"fmt"

	"os_model/internal/process"
	"os_model/internal/regulator"
)

// ---------------------------------------------------------------------------
// DirectiveKind — вид управляющей директивы оператора (лабораторная работа 9).
type DirectiveKind int

const (
	DirectivePause     DirectiveKind = iota // пауза моделирования
	DirectiveResume                         // продолжить моделирование
	DirectiveSpeedUp                        // увеличить скорость
	DirectiveSpeedDown                      // уменьшить скорость
	DirectiveQuit                           // выключить питание

	DirectiveAutoLoadPause  // приостановить автоматическую загрузку
	DirectiveAutoLoadResume // продолжить автоматическую загрузку
	DirectiveLoadTask       // загрузить новое задание
	DirectiveSetNextTask    // задать параметры следующего задания
	DirectiveClearNextTask  // сбросить переопределение параметров
	DirectiveSuspendProcess // приостановить выбранный процесс
	DirectiveResumeProcess  // активизировать выбранный процесс
	DirectiveDestroyProcess // уничтожить выбранный процесс
	DirectiveFinishAndQuit  // выполнить загруженные задания и выключить
	DirectiveSetMemory      // изменить размер памяти
)

// ---------------------------------------------------------------------------
// String возвращает имя директивы для индикации.
func (k DirectiveKind) String() string {
	switch k {
	case DirectivePause:
		return "пауза модели"
	case DirectiveResume:
		return "продолжить модель"
	case DirectiveSpeedUp:
		return "увеличить скорость"
	case DirectiveSpeedDown:
		return "уменьшить скорость"
	case DirectiveQuit:
		return "выключить питание"
	case DirectiveAutoLoadPause:
		return "пауза автозагрузки"
	case DirectiveAutoLoadResume:
		return "возобновить автозагрузку"
	case DirectiveLoadTask:
		return "загрузить задание"
	case DirectiveSetNextTask:
		return "параметры следующего задания"
```

```
	case DirectiveClearNextTask:
		return "сбросить параметры задания"
	case DirectiveSuspendProcess:
		return "приостановить процесс"
	case DirectiveResumeProcess:
		return "активизировать процесс"
	case DirectiveDestroyProcess:
		return "уничтожить процесс"
	case DirectiveFinishAndQuit:
		return "выполнить и выключить"
	case DirectiveSetMemory:
		return "изменить память"
	default:
		return "неизвестная директива"
	}
}

// ----------------------------------------------------------------------------
// Directive — директива оператора вместе с параметрами.
type Directive struct {
	Kind       DirectiveKind
	ProcessID  int
	Task       *process.Task
	MemorySize int
}

// ----------------------------------------------------------------------------
// DirectiveResult — результат выполнения последней директивы.
type DirectiveResult struct {
	Time    int    `json:"time"`
	Name    string `json:"name"`
	OK      bool   `json:"ok"`
	Message string `json:"message"`
}

// ----------------------------------------------------------------------------
// applyDirective выполняет одну директиву и запоминает результат.
func (sys *System) applyDirective(d Directive) {
	result := DirectiveResult{
		Time: sys.Time,
		Name: d.Kind.String(),
		OK:   true,
	}

	switch d.Kind {
	case DirectivePause:
		sys.Paused = true
		result.Message = "моделирование приостановлено"

	case DirectiveResume:
		sys.Paused = false
		result.Message = "моделирование продолжено"

	case DirectiveSpeedUp:
		sys.changeSpeed(speedStepPercent)
		result.Message = fmt.Sprintf("скорость %.1f такт/с", sys.Speed)

	case DirectiveSpeedDown:
		sys.changeSpeed(-speedStepPercent)
		result.Message = fmt.Sprintf("скорость %.1f такт/с", sys.Speed)

	case DirectiveQuit:
```

```
			sys.Quit = true
			result.Message = "питание выключено"

		case DirectiveAutoLoadPause:
			sys.AutoLoadEnabled = false
			result.Message = "автоматическая загрузка приостановлена"

		case DirectiveAutoLoadResume:
			sys.AutoLoadEnabled = true
			result.Message = "автоматическая загрузка возобновлена"

		case DirectiveLoadTask:
			if sys.GenerateAndLoadTask() {
				result.Message = fmt.Sprintf("задание #%d загружено", sys.LastTask.Task.ID)
			} else {
				result.OK = false
				result.Message = "задание не загружено: " + sys.LastTask.Reason
			}

		case DirectiveSetNextTask:
			if d.Task == nil {
				result.OK = false
				result.Message = "не заданы параметры задания"
				break
			}
			if err := validateTask(*d.Task); err != nil {
				result.OK = false
				result.Message = err.Error()
				break
			}
			task := *d.Task
			sys.NextTask = &task
			result.Message = fmt.Sprintf("следующее задание: размер %d, команд %d, ВВ %d %%,
приоритет %d",
				task.Size, task.TotalCommands, task.IOPercent, task.BasePriority)

		case DirectiveClearNextTask:
			sys.NextTask = nil
			result.Message = "параметры следующего задания сброшены"

		case DirectiveSuspendProcess:
			if err := sys.suspendProcess(d.ProcessID); err != nil {
				result.OK = false
				result.Message = err.Error()
			} else {
				result.Message = fmt.Sprintf("процесс #%d приостановлен", d.ProcessID)
			}

		case DirectiveResumeProcess:
			if err := sys.resumeProcess(d.ProcessID); err != nil {
				result.OK = false
				result.Message = err.Error()
			} else {
				result.Message = fmt.Sprintf("процесс #%d возобновлён", d.ProcessID)
			}

		case DirectiveDestroyProcess:
			if err := sys.destroyProcess(d.ProcessID); err != nil {
				result.OK = false
				result.Message = err.Error()
			} else {
				result.Message = fmt.Sprintf("процесс #%d уничтожен", d.ProcessID)
```

```
                }

        case DirectiveFinishAndQuit:
                sys.FinishAndQuit = true
                sys.AutoLoadEnabled = false
                sys.Paused = false
                sys.Regulator.ResumeAll()
                result.Message = "загруженные задания выполняются, новые не загружаются"

        case DirectiveSetMemory:
                if err := sys.Memory.SetTotalSize(d.MemorySize); err != nil {
                        result.OK = false
                        result.Message = err.Error()
                } else {
                        sys.Config.MemorySize = d.MemorySize
                        result.Message = fmt.Sprintf("размер памяти: %d единиц", d.MemorySize)
                }

        default:
                result.OK = false
                result.Message = "неизвестная директива"
        }

        sys.LastDirective = result
        sys.notify()
}

// ---------------------------------------------------------------------------
// validateTask проверяет параметры задания, заданные оператором.
func validateTask(task process.Task) error {
        if task.Size <= 0 {
                return fmt.Errorf("размер задания должен быть больше 0")
        }
        if task.TotalCommands <= 0 {
                return fmt.Errorf("число команд должно быть больше 0")
        }
        if task.IOPercent < 0 || task.IOPercent > 100 {
                return fmt.Errorf("процент команд ВВ должен быть в пределах 0..100")
        }
        if task.BasePriority <= 0 {
                return fmt.Errorf("базовый приоритет должен быть больше 0")
        }
        return nil
}

// ---------------------------------------------------------------------------
// findProcess ищет резидентный процесс по идентификатору.
func (sys *System) findProcess(id int) *process.PSW {
        for _, p := range sys.Regulator.Table {
                if p.ID == id {
                        return p
                }
        }
        return nil
}

// ---------------------------------------------------------------------------
// suspendProcess приостанавливает готовый или активный процесс.
func (sys *System) suspendProcess(id int) error {
        p := sys.findProcess(id)
        if p == nil {
                return fmt.Errorf("процесс #%d не найден", id)
```

```
	}
	if p.State != process.StateReady && p.State != process.StateActive {
		return fmt.Errorf("процесс #%d нельзя приостановить в состоянии %s",
			id, process.StateName(p.State))
	}
	if !sys.Regulator.Process(p, regulator.EventSuspend) {
		return fmt.Errorf("не удалось приостановить процесс #%d", id)
	}
	if sys.CPU.ActiveProcess == p {
		sys.CPU.Release()
	}
	return nil
}

// ---------------------------------------------------------------------------
// resumeProcess возвращает приостановленный процесс в очередь готовности.
func (sys *System) resumeProcess(id int) error {
	p := sys.findProcess(id)
	if p == nil {
		return fmt.Errorf("процесс #%d не найден", id)
	}
	if p.State != process.StateSuspended {
		return fmt.Errorf("процесс #%d не приостановлен", id)
	}
	if !sys.Regulator.Process(p, regulator.EventResume) {
		return fmt.Errorf("не удалось возобновить процесс #%d", id)
	}
	return nil
}

// ---------------------------------------------------------------------------
// destroyProcess уничтожает процесс, освобождая ЦПр, ВВ и память.
func (sys *System) destroyProcess(id int) error {
	p := sys.findProcess(id)
	if p == nil {
		return fmt.Errorf("процесс #%d не найден", id)
	}

	if sys.CPU.ActiveProcess == p {
		sys.CPU.Release()
	}
	sys.IO.Cancel(p)
	sys.Interrupts.Cancel(p)
	if !sys.Regulator.Destroy(p) {
		return fmt.Errorf("не удалось уничтожить процесс #%d", id)
	}
	return nil
}

// ---------------------------------------------------------------------------
// checkFinish завершает моделирование, когда все загруженные задания
// выполнены или уничтожены.
func (sys *System) checkFinish() {
	if sys.FinishAndQuit && len(sys.Regulator.Table) == 0 {
		sys.Quit = true
	}
}
```

internal/regulator/regulator.go

```
// Package regulator реализует регулировщик — часть ОС, отвечающую за
// состояния процессов, и включает планировщик (Planner).
//
```

```
// Только регулировщик имеет право изменять состояния процессов: остальные
// модули модели (центральный процессор, подсистема ввода-вывода) лишь
// обращаются к подпрограмме Process, сообщая о произошедшем событии.
// Планировщик решает одну задачу — выбор следующего процесса из списка
// готовности; состояния он не изменяет.
// Регулировщик переводит процесс в новое состояние и выполняет связанные
// с этим изменения общих данных модели: очереди готовности, таблицы слов
// состояний процессов, памяти и счётчика завершённых заданий.
package regulator

import (
	"os_model/internal/memory"
	"os_model/internal/process"
	"os_model/internal/stats"
)

// ---------------------------------------------------------------------------
// Event — событие модели, по которому регулировщик изменяет состояние процесса.
type Event int

// ---------------------------------------------------------------------------
const (
	EventLoad      Event = iota // загрузка задания: создан процесс в состоянии «Готов»
	EventActivate               // выбор планировщика: «Готов» → «Активен»
	EventIORequest              // инициализация ВВ: «Активен» → «Инициализация IO»
	EventIOStart                // процессор ВВ начал работу: «Инициализация IO» → «Блокирован»
	EventIOComplete             // прерывание от ВВ: «Блокирован» → «Готов»
	EventTerminate              // завершение задания: выгрузка процесса
	EventSuspend                // оператор приостановил процесс
	EventResume                 // оператор возобновил процесс
)

// ---------------------------------------------------------------------------
// Regulator — регулировщик модели. Включает планировщик (Planner),
// выбирающий следующий процесс из очереди готовности.
type Regulator struct {
	Planner *Planner // планировщик: очередь готовности и диспетчеризация

	Memory      *memory.Allocator // супервизор памяти: освобождается при завершении
	MaxProcesses int              // число записей в таблице слов состояний процессов

	// Stats — модуль статистики: накапливает входные и выходные параметры
	// процессов (лабораторная работа 6). Может быть nil.
	Stats *stats.Stats

	// Costs — длительности системных действий (лабораторная работа 7).
	Costs Costs

	Table     []*process.PSW // таблица слов состояний процессов (резидентные процессы)
	Completed int            // число выполненных (завершённых) заданий
	Destroyed int            // число процессов, уничтоженных оператором
}

// ---------------------------------------------------------------------------
// Process — основная подпрограмма регулировщика: обрабатывает событие event
// для процесса p, изменяя его состояние и общие данные модели.
// Возвращает false, если событие недопустимо для текущего состояния процесса.
func (r *Regulator) Process(p *process.PSW, event Event) bool {
	switch event {
	case EventLoad:
		// Загрузка задания: нужна свободная запись в таблице PSW.
		if !r.FreeSlot() {
```

```
                return false
        }
        // Системные затраты на загрузку начисляются уже готовым процессам;
        // новый процесс в таблицу ещё не добавлен и это время не получает.
        r.charge(stats.SystemLoad, r.Costs.Load)
        p.LoadTicks += r.Costs.Load
        p.State = process.StateReady
        r.Table = append(r.Table, p)
        r.Planner.Add(p)
        if r.Stats != nil {
                r.Stats.OnLoad(p)
        }

case EventActivate:
        // Планировщик выбрал процесс для выполнения на процессоре.
        if p.State != process.StateReady {
                return false
        }
        p.State = process.StateActive
        r.Planner.Remove(p)
        // Время выбора на ЦПр получают остальные готовые процессы;
        // выбранный процесс уже переведён в «Активен».
        r.charge(stats.SystemDispatch, r.Costs.Dispatch)
        p.SwitchTicks += r.Costs.Dispatch
        p.DispatchCount++

case EventIORequest:
        // Задание инициализирует операцию ввода-вывода: слово состояния
        // процесса сохраняется, процесс переводится в «Инициализация IO».
        if p.State != process.StateActive {
                return false
        }
        p.State = process.StateInitIO
        r.charge(stats.SystemIOInit, r.Costs.IOInit)
        p.IOInitTicks += r.Costs.IOInit
        p.IORequestCount++

case EventIOStart:
        // Процессор ВВ приступил к выполнению операции.
        if p.State != process.StateInitIO {
                return false
        }
        p.State = process.StateBlockedIO

case EventIOComplete:
        // Сигнал прерывания от процессора ВВ: процесс снова готов
        // к выполнению и помещается в очередь готовности.
        if p.State != process.StateBlockedIO {
                return false
        }
        // Прерывание обслуживается, пока процесс ещё «Блокирован»,
        // поэтому собственное время ожидания он не получает.
        r.charge(stats.SystemIOComplete, r.Costs.IOInterrupt)
        p.IOIntrTicks += r.Costs.IOInterrupt
        p.IOCompleteCount++
        p.State = process.StateReady
        r.Planner.Add(p)

case EventTerminate:
        // Завершение задания: выгрузка процесса.
        if p.State != process.StateActive {
                return false
```

```
                }
                r.charge(stats.SystemTerminate, r.Costs.Terminate)
                p.TermTicks += r.Costs.Terminate
                r.Planner.Remove(p)
                r.Memory.Free(*p.Task)
                r.removeFromTable(p)
                p.State = process.StateNone
                r.Completed++
                if r.Stats != nil {
                        r.Stats.OnComplete(p)
                }

        case EventSuspend:
                // Приостановить можно готовый или активный процесс. Если процесс
                // активен, освобождение ЦПр выполняет ядро модели.
                if p.State != process.StateReady && p.State != process.StateActive {
                        return false
                }
                if p.State == process.StateReady {
                        r.Planner.Remove(p)
                }
                p.State = process.StateSuspended

        case EventResume:
                // Возобновлённый процесс снова становится готовым к выполнению.
                if p.State != process.StateSuspended {
                        return false
                }
                p.State = process.StateReady
                r.Planner.Add(p)

        default:
                return false
        }

        return true
}

// ---------------------------------------------------------------------------
// IncreaseReadyTime — подпрограмма «Увеличить время готовых процессов»
// (лабораторная работа 7). Увеличивает на ticks время пребывания в системе
// всех процессов, находящихся в состоянии «Готов». Именно они ожидают,
// пока ОС выполняет системное действие.
//
// Состояния «Активен», «Инициализация IO» и «Блокирован» не изменяются:
// активный процесс в это время не ожидает, а процесс, блокированный
// по вводу-выводу, по правилу лабораторной работы системное время
// не получает.
func (r *Regulator) IncreaseReadyTime(ticks int) {
        if ticks <= 0 {
                return
        }
        for _, p := range r.Table {
                if p.State == process.StateReady {
                        p.SystemWaitTicks += ticks
                }
        }
}

// ---------------------------------------------------------------------------
// charge начисляет время системного действия: увеличивает ожидание готовых
// процессов и суммарные системные затраты. Собственные затраты процесса
```

```
// (LoadTicks, SwitchTicks и т. д.) начисляет вызвавшая подпрограмма.
func (r *Regulator) charge(action stats.SystemAction, ticks int) {
	r.IncreaseReadyTime(ticks)
	if r.Stats != nil {
		r.Stats.OnSystemAction(action, ticks)
	}
}

// ----------------------------------------------------------------------------
// Terminate — подпрограмма завершения процесса, используемая процессором
// при выполнении команды завершения задания.
func (r *Regulator) Terminate(p *process.PSW) {
	r.Process(p, EventTerminate)
}

// ----------------------------------------------------------------------------
// Destroy — уничтожение процесса оператором. Процесс не считается
// выполненным: счётчик Completed не увеличивается, средние времена
// завершённых процессов не меняются.
func (r *Regulator) Destroy(p *process.PSW) bool {
	if p == nil || p.State == process.StateNone {
		return false
	}
	r.Planner.Remove(p)
	r.Memory.Free(*p.Task)
	r.removeFromTable(p)
	p.State = process.StateNone
	r.Destroyed++
	if r.Stats != nil {
		r.Stats.OnDestroy()
	}
	return true
}

// ----------------------------------------------------------------------------
// ResumeAll возобновляет все приостановленные процессы. Используется
// директивой «Выполнить загруженные задания и выключить питание».
func (r *Regulator) ResumeAll() {
	for _, p := range r.Table {
		if p.State == process.StateSuspended {
			p.State = process.StateReady
			r.Planner.Add(p)
		}
	}
}

// ----------------------------------------------------------------------------
// FreeSlot сообщает, есть ли свободная запись в таблице слов состояний процессов.
func (r *Regulator) FreeSlot() bool {
	return len(r.Table) < r.MaxProcesses
}

// ----------------------------------------------------------------------------
// removeFromTable удаляет запись процесса из таблицы слов состояний процессов.
func (r *Regulator) removeFromTable(p *process.PSW) {
	for i, t := range r.Table {
		if t == p {
			r.Table = append(r.Table[:i], r.Table[i+1:]...)
			return
		}
	}
}
```

5. Результаты проверки

Новые постоянные тесты не добавлялись; проверка выполнялась ручными прогонами и существующими тестами проекта.

№	Проверка	Ожидаемый результат	Фактический результат	Вывод
1	Пауза и возобновление автозагрузки	AutoLoadEnabled меняется	Пауза/возобновление работают	Выполнено
2	Параметры следующего задания	Загружается ровно заданное задание, затем сброс	Размер 123, параметры применились один раз	Выполнено
3	Приостановка активного процесса	Состояние «Приостановлен», ЦПр освобождён	Состояние 5, активный процесс снят	Выполнено
4	Возобновление процесса	Состояние «Готов», возврат в очередь	Состояние 1, процесс в очереди	Выполнено
5	Уничтожение blocked-процесса	Память и ВВ освобождены, Completed не растёт	Таблица 0, память 0, Destroyed 1, Completed 0	Выполнено
6	Изменение памяти	Уменьшение ниже занятой отклоняется	Запрос 50 отклонён, 8192 применён	Выполнено
7	Выполнить и выключить	Модель завершается после выполнения загруженных	Quit = true, Completed = 2	Выполнено
8	Существующие тесты и сборка	Все проходят	go test ./... — ok; wails build — Built	Выполнено

Команды проверки:

```
$ go test ./... -count=1
ok   os_model/internal/os
ok   os_model/internal/regulator
ok   os_model/internal/stats

$ wails build -tags webkit2_41
Built build/bin/os_model
```

6. Экранные формы

На рисунках 8–10 показаны экранные формы панели директив. В примере выбран процесс #1, приостановлен, автозагрузка выключена, для следующего задания заданы размер 250, 12 команд, 30 % ВВ и приоритет 7.

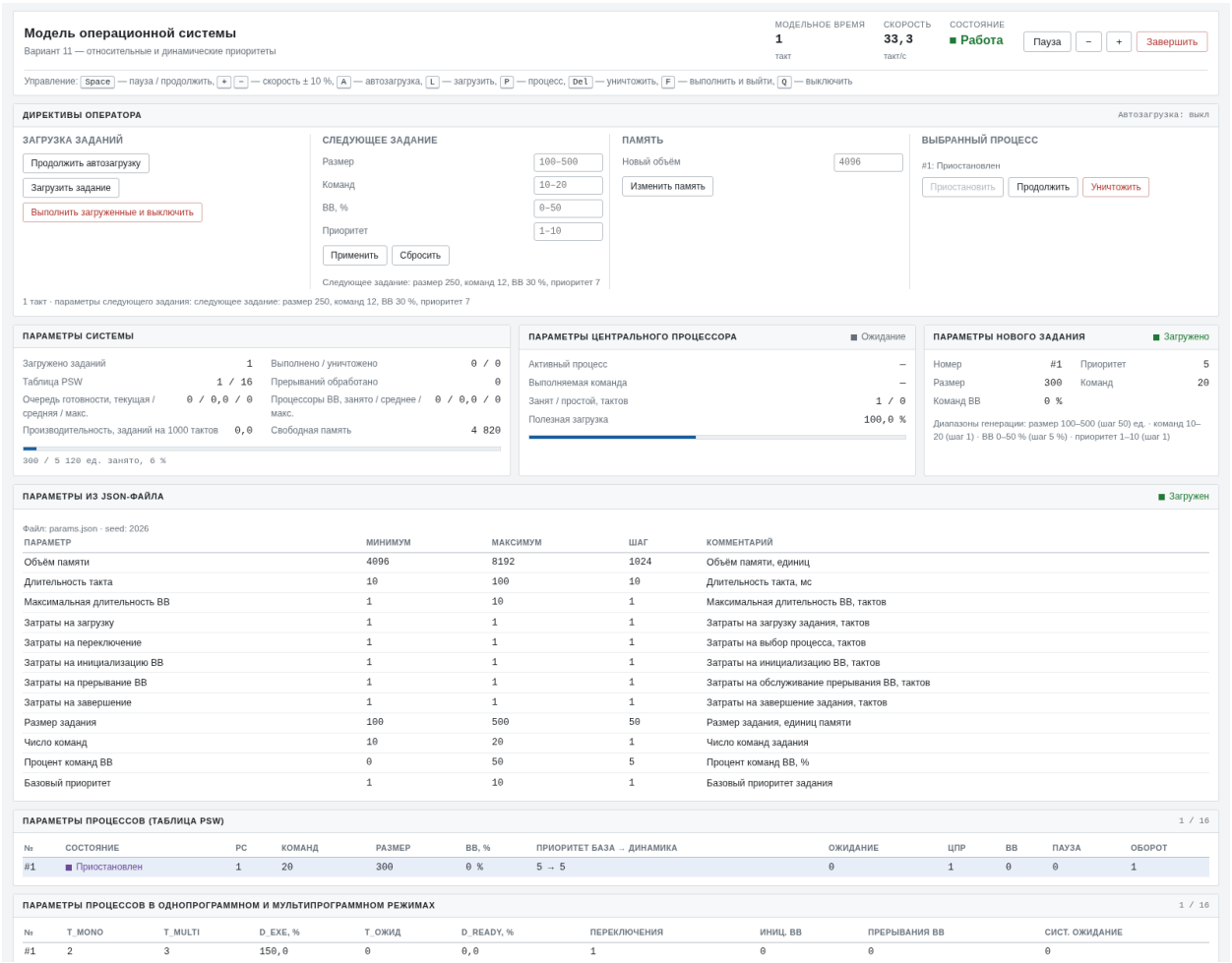


Рисунок 8 — Главный экран с панелью директив

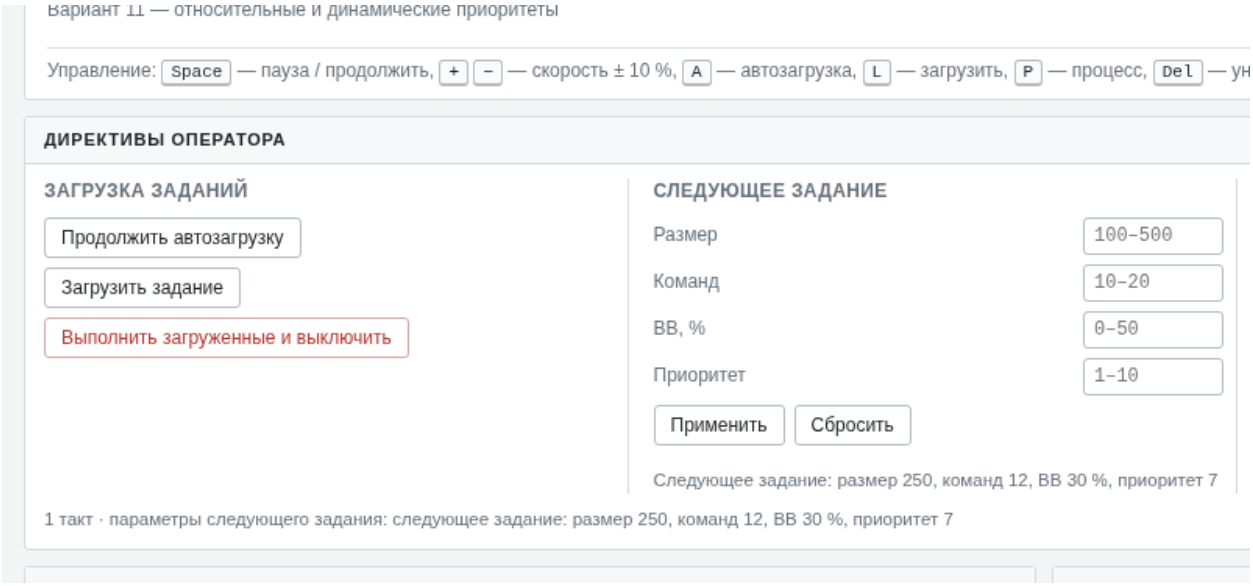


Рисунок 9 — Левая часть панели директив: загрузка и параметры следующего задания

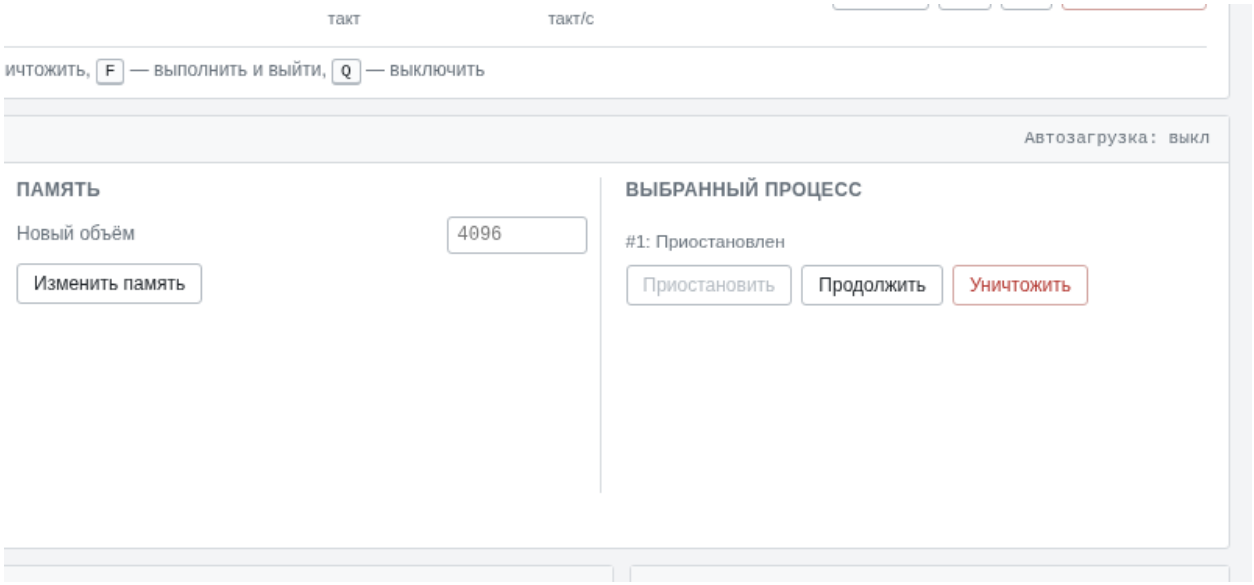


Рисунок 10 — Правая часть панели директив: память и выбранный процесс

В таблице PSW выбранный процесс подсвечен, отображается состояние «Приостановлен» и время в этом состоянии. Результат последней директивы выводится под панелью управления.

Выводы

В ходе лабораторной работы разработан язык управления моделью ОС. Реализованы директивы паузы и скорости, управления автозагрузкой, ручной загрузки, изменения параметров следующего задания, приостановки/возобновления и уничтожения выбранного процесса, изменения памяти, выполнения загруженных заданий с последующим выключением, а также seed RNG из командной строки.

Для поддержки директив добавлено состояние «Приостановлен», события регулировщика Suspend/Resume, отмена операций BB и прерываний при уничтожении, счётчик DestroyedCount. Интерфейс получил панель управления и выбор процесса по строке таблицы PSW. Проверки подтвердили корректность переходов состояний, освобождения ресурсов и завершения модели.